5 性能分析方法

当你在进行高级优化工作时，比如将更好的算法集成到应用程序中，通常很容易判断性能是否有所提高，因为基准测试结果很明显。大幅提速，如 2 倍、3 倍等，从性能分析的角度来看相对容易。当你从程序中消除了大量计算时，你预期会在运行时间上看到明显的差异。

但也有一些情况下，你会看到执行时间的微小变化，比如 5%，而你不知道这个变化是从哪里来的。仅仅依靠计时或吞吐量测量并不能解释性能是为什么提高或下降的。在这种情况下，我们需要进行性能分析，以了解我们观察到的减速或加速的潜在原因。

性能分析类似于侦探工作。要解决性能谜团，你需要收集尽可能多的数据，然后尝试形成一个假设。一旦假设被提出，你就设计一个实验来证明或反驳它。这可能需要多次来回，直到你找到线索。就像一个优秀的侦探一样，你会尽力收集尽可能多的证据来证实或反驳你的假设。一旦你有足够的线索，你就可以对你观察到的行为做出令人信服的解释。

当你刚开始处理性能问题时，你可能只有测量数据，比如在代码变更之前和之后的数据。基于这些测量数据，你得出结论：程序变慢了X个百分点。如果你知道减速发生在某次提交之后，这可能已经为你提供了足够的信息来修复问题。但如果你没有良好的参考点，那么减速的可能原因就是无穷无尽的，你需要收集更多的数据。收集这些数据的最流行方法之一是对应用程序进行性能分析并查看热点。本章介绍了这个方法以及其他一些在性能工程中被证明有效的方法。

接下来的问题是：“可用的性能数据有哪些，如何收集？”堆栈的硬件和软件层都有跟踪性能事件并记录它们的设施。在这个背景下，所谓的硬件是指执行程序的 CPU，而软件是指操作系统、库、应用程序本身和用于分析的其他工具。通常，软件堆栈提供高级指标，如时间、上下文切换次数和页面错误，而 CPU 则监视缓存未命中、分支错误预测和其他与 CPU 相关的事件。根据你试图解决的问题，一些指标比其他指标更有用。因此，这并不意味着硬件指标总会给我们提供程序执行的更精确的概览。例如，一些指标，比如上下文切换次数，CPU 无法提供。性能分析工具，如 Linux Perf，可以使用来自操作系统和 CPU 的数据。

正如你可能猜到的那样，性能工程师可能会使用数百种数据源。由于本书是关于 CPU 底层性能的，我们将重点介绍收集硬件级别信息的技术。我们将介绍一些最流行的性能分析技术：代码插桩、跟踪、特性化、采样和 roofline 模型。我们还将讨论静态性能分析技术和不涉及运行实际应用程序的编译器优化报告。

5.1 代码插桩 (Instrumentation)

也许有史以来进行性能分析的第一种方法就是代码 插桩 (Instrumentation)[57]。它是一种在程序中插入额外代码以收集特定运行时信息的技术。Listing 5.1展示了在函数开头插入`printf`语句的最简单示例，以指示该函数何时被调用。然后，运行程序并计算输出中看到“foo 被调用”的次数。也许，世界上每个程序员在其职业生涯中至少有一次这样做过。

代码插桩

```
int foo(int x) {
+ printf("foo被调用\n");
  // 函数体...
}
```

[57] 测试领域内所用的 Instrumentation 术语翻译为”插桩”

行首的加号表示此行是添加的，不在原始代码中。通常，插桩代码并不意味着将其推送到代码库中，而是用于收集所需的数据，然后可以丢弃。

稍微有趣一些的代码插桩示例在 Listing 5.1中给出。在这个虚构的代码示例中，函数`findObject`在地图上搜索具有某些属性`p`的对象的坐标。函数`findObj`返回使用当前坐标`c`定位正确对象的置信度级别。如果是完全匹配，我们停止搜索循环并返回坐标。如果置信度高于`threshold`，我们选择`zoomIn`以找到对象更精确的位置。否则，我们在`searchRadius`范围内获取新的坐标以便下次尝试搜索。

插桩代码由两个类组成：`histogram`和`incrementor`。前者跟踪我们感兴趣的变量值及其出现频率，然后在程序完成后打印直方图。后者只是一个辅助类，用于将值推送到`histogram`对象中。它非常简单，可以快速调整以满足您的特定需求。我有一个稍微更高级的版本，通常会将其复制粘贴到我正在工作的任何项目中，然后将其删除。

代码：代码插桩

```
+ struct histogram {
+   std::map<uint32_t, std::map<uint32_t, uint64_t>> hist;
+   ~histogram() {
+     for (auto& tripCount : hist)
+       for (auto& zoomCount : tripCount.second)
+         std::cout << "[" << tripCount.first << "]["
+                   << zoomCount.first << "] :  "
+                   << zoomCount.second << "\n";
+   }
+ };
+ histogram h;

+ struct incrementor {
+   uint32_t tripCount = 0;
+   uint32_t zoomCount = 0;
+   ~incrementor() {
+      h.hist[tripCount][zoomCount]++;
+   }
+ };

Coords findObject(const ObjParams& p, Coords c, float searchRadius) {
+ incrementor inc;
  while (true) {
+   inc.tripCount++;
    float match = findObj(c, p);
    if (exactMatch(match))
      return c;
    if (match > threshold) {
      searchRadius = zoomIn(c, searchRadius);
+     inc.zoomCount++;
    }
    c = getNewCoords(searchRadius);
  }
  return c;
```

```
}
```

在这个假设情景中，我们添加了插桩代码以了解在找到对象之前我们多频繁地zoomIn。变量inc.tripCount计算循环退出之前循环运行的次数，而变量inc.zoomCount计算我们减少搜索半径的次数。我们总是期望inc.zoomCount小于或等于inc.tripCount。下面是运行插桩程序后可能观察到的输出：

```
[7][6]:  2
[7][5]:  6
[7][4]:  20
[7][3]:  156
[7][2]:  967
[7][1]:  3685
[7][0]:  251004
[6][5]:  2
[6][4]:  7
[6][3]:  39
[6][2]:  300
[6][1]:  1235
[6][0]:  91731
[5][4]:  9
[5][3]:  32
[5][2]:  160
[5][1]:  764
[5][0]:  34142
[4][4]:  5
[4][3]:  31
[4][2]:  103
[4][1]:  195
[4][0]:  14575
...
```

在方括号中的第一个数字是循环的次数，第二个数字是在同一个循环中进行的zoomIn次数。冒号后面的数字是该特定组合的出现次数。例如，我们观察到 7 次循环迭代和 6 次zoomIn发生了两次，循环运行了 7 次迭代且没有zoomIn的情况发生了 251004 次，依此类推。然后，您可以绘制数据以进行更好的可视化，采用一些其他统计方法，但我们可以得出的主要观点是zoomIn并不频繁。在调用了 400k 次findObject的情况下，总共有 10k 次zoomIn调用。

本书的后续章节包含许多示例，说明了这类信息如何用于基于数据的优化。在我们的情况下，我们得出结论：findObj经常无法找到对象。这意味着循环的下一次迭代将尝试使用新坐标来找到对象，但搜索半径仍然相同。有了这个信息，我们可以尝试一些优化：1）并行运行多个搜索，并在其中任何一个成功时同步；2）为当前搜索区域预先计算某些内容，从而消除findObj内的重复工作；3）编写一个软件流水线，调用getNewCoords以生成下一组所需坐标，并从内存中预取相应的地图位置。本书的第二部分将更深入地探讨一些这样的技术。

代码插桩在需要关于程序执行的特定知识时提供了非常详细的信息。它允许我们跟踪程序中每个变量的任何信息。在优化大型代码块时，使用这种方法通常会产生最好的见解，因为您可以使用自上而下的方法（插桩主函数，然后逐步深入到其被调用的函数）来定位性能问题。虽然代码插桩在小程序的情况下并不是很有帮助，但通过让开发人员观察应用程序的架构和流程，它提供了最大的价值和见解。对于与不熟悉的代码库一起工作的人来说，这种技术尤其有帮助。

值得一提的是，代码插桩在具有许多不同组件的复杂系统中表现突出，这些组件根据输入或时间的不同而产生不同的反应。例如，在游戏中，通常有一个渲染线程、一个物理线程、一个动画线程等。对这样的大型模块进行插桩有助于相对快速地理解哪个模块是问题的源头。因为有时，优化不仅仅是优化代码，还包括数据。例如，渲染可能太慢是因为网格未压缩，或者物理可能太慢是因为场景中的对象太多。

插桩技术在实时场景的性能分析中被广泛使用，例如视频游戏和嵌入式开发。一些性能分析器将插桩与其他技术（如跟踪和采样）混合在一起。我们将在 Section 7.8中介绍其中一个混合性能分析器，名为 Tracy。

虽然在许多情况下代码插桩是强大的，但它并不提供有关代码如何从操作系统或 CPU 的角度执行的任何信息。例如，它无法告诉您进程被调度到执行中和退出执行的频率（由操作系统知道），或者分支错误预测发生的次数（由 CPU 知道）。被插桩的代码是应用程序的一部分，并具有与应用程序本身相同的特权。它在用户空间中运行，无法访问内核。

但更重要的是，这种技术的缺点是每次需要插桩新内容，例如另一个变量时，都需要重新编译。这可能会给工程师带来负担，并增加分析时间。不幸的是，还有其他一些缺点。由于通常您关心的是应用程序中的热点路径，因此您正在为位于代码性能关键部分的内容进行插桩。在热点路径中注入插桩代码可能很容易导致整体基准测试减慢 2 倍。请记住不要对被插桩的程序进行基准测试，即不要在同一运行中进行评分和分析。请记住，通过对代码进行插桩，您会改变程序的行为，因此您可能看不到之前看到的相同效果。

上述所有内容增加了实验之间的时间，消耗了更多的开发时间，这就是为什么工程师如今很少手动插桩他们的代码的原因。然而，自动化代码插桩仍然被编译器广泛使用。编译器能够自动对整个程序进行插桩，并收集有关执行的有趣统计信息。自动插桩最广泛的用例是代码覆盖分析和基于性能指导的优化（参见 Section 11.7）。

在谈到插桩时，重要的是要提到二进制插桩技术。二进制插桩的思想类似，但它是在已构建的可执行文件上完成的，而不是在源代码级别上。有两种类型的二进制插桩：静态（在构建之前完成）和动态（在程序执行时根据需要插入插桩代码）。动态二进制插桩的主要优势在于它不需要重新编译和重新链接程序。此外，通过动态插桩，可以将插桩的量限制为仅限于感兴趣的代码区域，而不是整个程序。

二进制插桩在性能分析和调试中非常有用。二进制插桩最流行的工具之一是 Intel Pin[58]工具。Pin 拦截程序在发生有趣事件时的执行，并生成从程序中的这一点开始的新插桩代码。它允许收集各种运行时信息，例如：

- 指令计数和函数调用计数。
- 拦截函数调用和应用程序中任何指令的执行。
- 允许通过在区域开始时捕获内存和硬件寄存器状态来“记录和重放”程序区域。

与代码插桩类似，二进制插桩只允许对用户级代码进行插桩，而且可能非常慢。

5.2 跟踪

跟踪在概念上与插桩非常相似，但略有不同。代码插桩假设用户可以编排他们应用程序的代码。另一方面，跟踪依赖于程序的外部依赖项的现有插桩。例如，`strace`工具使我们能够跟踪系统调用，并可以被视为对 Linux 内核的插桩。英特尔处理器跟踪（见附录 D）使您能够记录程序执行的指令，并可以被视为对 CPU 的插桩。跟踪可以从事先适当插桩的组件中获得，并且不受更改的影响。跟踪通常被用作黑匣子方法，其中用户无法修改应用程序的代码，但他们希望了解程序在幕后执行的操作。

Listing 5.2提供了使用 Linux `strace`工具跟踪系统调用的示例，显示了运行`git status`命令时输出的前几行。通过使用`strace`跟踪系统调用，可以得知每个系统调用的时间戳（最左边的列），其退出状态以及每个系统调用的持续时间（在尖括号内）。

代码清单：使用 strace 跟踪系统调用。

[58] PIN - https://software.intel.com/en-us/articles/pin-a-dynamic-binary-instrumentation-tool

```
$ strace -tt -T -- git status
17:46:16.798861 execve("/usr/bin/git", ["git", "status"], 0x7ffe705dcd78
                /* 75 vars */) = 0 <0.000300>
17:46:16.799493 brk(NULL)               = 0x55f81d929000 <0.000062>
17:46:16.799692 access("/etc/ld.so.nohwcap", F_OK) = -1 ENOENT
                (No such file or directory) <0.000063>
17:46:16.799863 access("/etc/ld.so.preload", R_OK) = -1 ENOENT
                (No such file or directory) <0.000074>
17:46:16.800032 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
                <0.000072>
17:46:16.800255 fstat(3, {st_mode=S_IFREG|0644, st_size=144852, ...}) = 0
                <0.000058>
17:46:16.800408 mmap(NULL, 144852, PROT_READ, MAP_PRIVATE, 3, 0)
                = 0x7f6ea7e48000 <0.000066>
17:46:16.800619 close(3)                = 0 <0.000123>
...
```

跟踪的开销非常取决于我们尝试跟踪的内容。例如，如果我们跟踪的程序几乎不进行系统调用，那么在strace下运行它的开销将接近零。另一方面，如果我们跟踪的程序严重依赖于系统调用，那么开销可能会非常大，例如，增加了 100 倍[59]。此外，跟踪可能会生成大量数据，因为它不会跳过任何样本。为了补偿这一点，跟踪工具提供了过滤器，使您能够将数据收集限制为特定的时间片段或特定代码段。

通常，类似于插桩的跟踪用于探查系统中的异常情况。例如，您可能想要确定在程序出现 10 秒不响应的情况下应用程序中发生了什么。正如您将在后面看到的，采样方法并不是为此设计的，但是通过跟踪，您可以看到是什么导致了程序不响应。例如，使用英特尔 PT，您可以重构程序的控制流并确切地知道执行了哪些指令。

跟踪对调试也非常有用。其底层特性支持基于记录的跟踪的“记录和重放”用例。Mozilla 的一个这样的工具是rr[60]调试器，它执行进程的记录和重放，支持向后单步执行等等。大多数跟踪工具都能够为事件添加时间戳，这使我们能够与在那段时间内发生的外部事件进行相关。也就是说，当我们观察到程序中出现故障时，我们可以查看我们应用程序的跟踪，并将此故障与在该时间段内整个系统中发生的情况进行关联。

5.3 工作负载特征化

工作负载特征化是通过定量参数和函数描述工作负载的过程。简单来说，它意味着计算某些性能事件的绝对数量。特征化的目标是定义工作负载的行为并提取其最重要的特征。在高层次上，一个应用程序可以属于以下一种或多种类型：交互式、数据库、实时、基于网络的、大规模并行等。不同的工作负载可以使用不同的指标和参数来解决特定的应用程序领域。

这是一本关于低级性能的书，记住了吗？所以，我们将专注于提取与 CPU 和内存性能相关的特征。从 CPU 角度看，最好的工作负载特征化示例是顶层微体系结构分析（Top-down Microarchitecture Analysis，TMA）方法，我们将在 Section 6.1中仔细研究。TMA 试图通过将应用程序放入以下 4 个桶之一来表征其性能：CPU 前端、CPU 后端、退役 (retired) 和错误预测，具体取决于造成最多性能问题的原因。TMA 使用性能监视计数器（PMCs）收集所需信息，并识别 CPU 微体系结构的低效使用。

但即使没有完全成熟的特征化方法，收集某些性能事件的绝对数量也可能会有所帮助。我们希望前一章中对四种不同基准测试的性能指标进行的案例研究证明了这一点。PMCs 是低级性能分析的非常重要的工具。它们可以提供有

[59] 一篇有关strace的文章，作者是 B. Gregg - http://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html

[60] Mozilla rr 调试器 - https://rr-project.org/

关程序执行的独特信息。PMCs 通常以两种模式使用：“计数”和“采样”。计数模式用于工作负载特征化，而采样模式用于查找热点，我们将很快讨论。

5.3.1 计数性能事件

计数背后的想法非常简单：我们希望在程序运行时计数某些性能事件的绝对数量。图 29 展示了从程序开始到结束计数性能事件的过程。

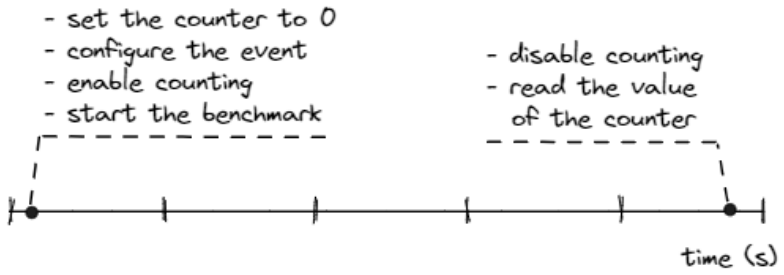


Figure 29: 计数性能事件。

图 29 中概述的步骤大致代表了典型分析工具会执行的操作来计数性能事件。这个过程是在`perf stat`工具中实现的，它可以用于计数各种硬件事件，比如指令数、周期数、缓存失效等。下面是`perf stat`的输出示例：

```
$ perf stat -- ./a.exe
 10580290629  cycles          #    3,677 GHz
  8067576938  instructions    #    0,76  insn per cycle
  3005772086  branches        # 1044,472 M/sec
   239298395  branch-misses   #    7,96% of all branches
```

知道这些数据非常有用。首先，它使我们能够快速发现一些异常，比如高的分支误预测率或低的 IPC。此外，当您进行了代码更改并希望验证更改是否改进了性能时，它可能会派上用场。查看绝对数值可能有助于您证明或拒绝代码更改。主要作者将`perf stat`用作简单的基准包装器。由于计数事件的开销很小，几乎所有基准测试都可以自动在`perf stat`下运行。它作为性能调查的第一步。有时，可以立即发现异常，这可以节省一些分析时间。

5.3.2 手动收集性能计数器数据

现代 CPU 拥有数百个可计数的性能事件。记住所有这些事件及其含义是非常困难的。更难的是理解何时使用特定的 PMC。这就是为什么通常我们不建议手动收集特定的 PMCs，除非您真的知道自己在做什么。相反，我们建议使用像 Intel Vtune Profiler 这样的工具来自动化这个过程。尽管如此，有时候您可能有兴趣收集特定的 PMCs。

您可以在 [Intel, 2023b, 第 3B 卷，第 19 章] 中找到所有 Intel CPU 代数的性能事件的完整列表。PMCs 的描述也可以在perfmon-events.intel.com上找到。每个事件都使用`Event`和`Umask`十六进制值进行编码。有时性能事件还可以使用附加参数进行编码，例如`Cmask`、`Inv`等。表 [6] 显示了针对 Intel Skylake 微体系结构编码的两个性能事件的示例。

Table 6: Skylake 性能事件的编码示例。

事件编号	Umask 值	事件掩码助记符	描述
C0H	00H	INST_RETIRED. ANY_P	退役 (retired) 的指令数量。
C4H	00H	BR_INST_RETIRED. ALL_BRANCHESk	退役 (retired) 的分支指令。

Linux perf提供了常用性能计数器的映射。可以通过伪名称而不是指定Event和Umask十六进制值来访问它们。例如，branches只是BR_INST_RETIRED.ALL_BRANCHES的同义词，并且将测量相同的内容。可以使用perf list查看可用映射名称的列表：

```
$ perf list
  branches           [Hardware event]
  branch-misses      [Hardware event]
  bus-cycles         [Hardware event]
  cache-misses       [Hardware event]
  cycles             [Hardware event]
  instructions       [Hardware event]
  ref-cycles         [Hardware event]
```

但是，Linux perf并不为每个 CPU 架构的所有性能计数器提供映射。如果您要查找的 PMC 没有映射，则可以使用以下语法进行收集：

```
$ perf stat -e cpu/event=0xc4,umask=0x0,name=BR_INST_RETIRED.ALL_BRANCHES/ -- ./a.exe
```

由于访问 PMCs 需要 root 访问权限，因此并非每个环境都可以使用性能计数器。在虚拟化环境中运行的应用程序通常没有 root 访问权限。对于在公共云中执行的程序，如果虚拟机（VM）管理器未正确向客户端公开 PMU 编程接口，则在客户端容器中直接运行基于 PMU 的分析器将不会产生有用的输出。因此，基于 CPU 性能计数器的分析器在虚拟化和云环境中效果不佳 [Du et al., 2010]。尽管情况正在改善。VmWare® 是第一个启用[61]虚拟 CPU 性能计数器（vPMC）的 VM 管理器之一。AWS EC2 云[62]为专用主机启用了 PMCs。

5.3.3 多路复用和事件缩放

有些情况下，我们希望同时计数许多不同的事件。但是只有一个计数器，一次只能计数一件事情。这就是为什么 PMU 中有多个计数器的原因（在最近的英特尔 Goldencove 微体系结构中，每个硬件线程有 12 个可编程的 PMC，每个线程有 6 个）。即使这样，固定和可编程计数器的数量并不总是足够的。Top-down 微体系结构分析（TMA）方法要求在单个程序执行中收集多达 100 种不同的性能事件。现代 CPU 没有那么多的计数器，这就是多路复用发挥作用的时候。

如果事件比计数器多，分析工具使用时间多路复用为每个事件提供访问监视硬件的机会。图 30a 显示了只有 4 个 PMC 可用时 8 个性能事件之间的多路复用示例。

通过多路复用，事件并不是一直被测量的，而只在一段时间内被测量。在运行结束时，性能分析工具需要根据总启用时间对原始计数进行缩放：

$$\text{最终计数} = \text{原始计数} \times (\text{运行时间}/\text{启用时间})$$

让我们以图 30b 为例。假设在分析过程中，我们能够在三个时间间隔内测量来自第 1 组的一个事件。每个测量间隔持续 100ms（启用时间）。程序运行时间为 500ms（运行时间）。该计数器的总事件数为 10’000（原始计数）。因此，最终计数需要按以下方式进行缩放：

$$\text{最终计数} = 10'000 \times (500ms/(100ms \times 3)) = 16'666$$

这提供了在整个运行过程中测量事件时计数的估计。非常重要的是要理解，这仍然是一个估计值，而不是实际计数。多路复用和缩放可以安全地用于执行长时间间隔内相同代码的稳定工作负载。然而，如果程序经常在不同的热点之

[61] VMWare PMCs - https://www.vladan.fr/what-are-vmware-virtual-cpu-performance-monitoring-counters-vpmcs/

[62] Amazon EC2 PMCs - http://www.brendangregg.com/blog/2017-05-04/the-pmcs-of-ec2.html

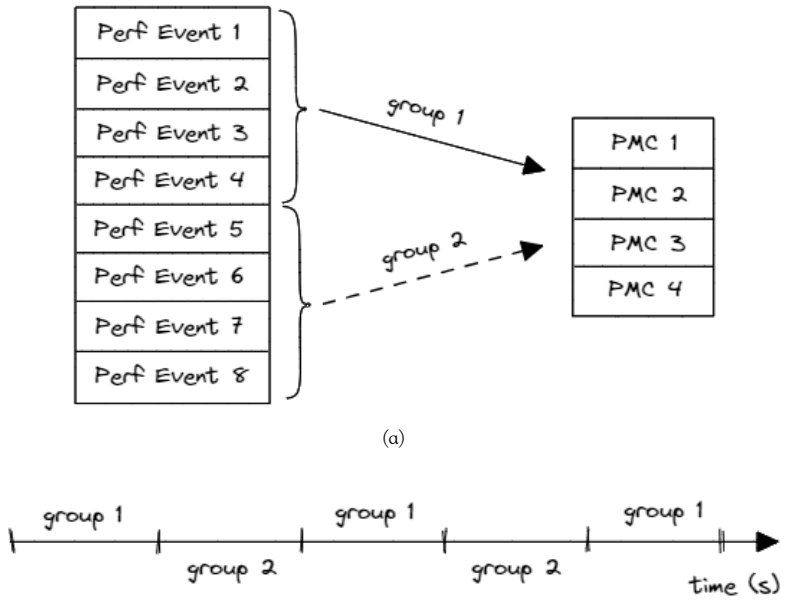


Figure 30: 8 个性能事件之间的多路复用示例，只有 4 个 PMC 可用。

间跳转，即有不同的阶段，那么就会产生盲点，这可能会在缩放过程中引入错误。为了避免缩放，可以尝试将事件的数量减少到不超过可用物理 PMC 的数量。然而，这将需要多次运行基准测试，以测量感兴趣的所有计数器。

5.4 使用标记器 API

在某些情况下，我们可能对分析特定代码区域的性能感兴趣，而不是整个应用程序。例如，当您开发一段新代码并只想关注该代码时，就会遇到这种情况。自然地，您会希望跟踪优化进度并捕获其他性能数据，以帮助您一路前进。大多数性能分析工具都提供特定的 标记器 API，可以让您做到这一点。这里有一些例子：

- Likwid 有 `LIKWID_MARKER_START / LIKWID_MARKER_STOP` 宏。
- Intel VTune 有 `__itt_task_begin / __itt_task_end` 函数。
- AMD uProf 有 `amdProfileResume / amdProfilePause` 函数。

这种混合方法结合了检测和性能事件计数的优点。标记器 API 允许我们将性能统计数据归因于代码区域（循环、函数）或功能片段（远程过程调用 (RPC)、输入事件等），而不是测量整个程序。您获得的数据质量足以证明这种努力是值得的。例如，在追查仅针对特定类型 RPC 出现的性能漏洞时，您可以仅针对该类型的 RPC 启用监控。

下面我们提供了一个非常基本的示例，展示了如何使用 libpfm4[63]，这是一个流行的用于收集性能监控事件的 Linux 库。它构建在 Linux `perf_events` 子系统之上，该子系统允许您直接访问性能事件计数器。`perf_events` 子系统相当底层，因此 `libfm4` 包在这里很有用，因为它增加了用于识别 CPU 上可用事件的发现工具以及围绕原始 `perf_event_open` 系统调用的包装库。Listing 64 展示了如何使用 `libpfm4` 为 C-Ray[64] benchmark 的 `render` 函数进行检测。

清单：在 C-Ray benchmark 上使用 libpfm4 标记器 API

```
+#include <perfmon/pfmlib.h>
+#include <perfmon/pfmlib_perf_event.h>
```

[63] libpfm4 - https://sourceforge.net/p/perfmon2/libpfm4/ci/master/tree/

[64] C-Ray 基准测试 - https://openbenchmarking.org/test/pts/c-ray

```
...
/* render a frame of xsz/ysz dimensions into the provided framebuffer */
void render(int xsz, int ysz, uint32_t *fb, int samples) {
   ...
+  pfm_initialize();
+  struct perf_event_attr perf_attr;
+  memset(&perf_attr, 0, sizeof(perf_attr));
+  perf_attr.size = sizeof(struct perf_event_attr);
+  perf_attr.read_format = PERF_FORMAT_TOTAL_TIME_ENABLED |
+                          PERF_FORMAT_TOTAL_TIME_RUNNING | PERF_FORMAT_GROUP;
+
+  pfm_perf_encode_arg_t arg;
+  memset(&arg, 0, sizeof(pfm_perf_encode_arg_t));
+  arg.size = sizeof(pfm_perf_encode_arg_t);
+  arg.attr = &perf_attr;
+
+  pfm_get_os_event_encoding("instructions", PFM_PLM3, PFM_OS_PERF_EVENT_EXT, &arg);
+  int leader_fd = perf_event_open(&perf_attr, 0, -1, -1, 0);
+  pfm_get_os_event_encoding("cycles", PFM_PLM3, PFM_OS_PERF_EVENT_EXT, &arg);
+  int event_fd = perf_event_open(&perf_attr, 0, -1, leader_fd, 0);
+  pfm_get_os_event_encoding("branches", PFM_PLM3, PFM_OS_PERF_EVENT_EXT, &arg);
+  event_fd = perf_event_open(&perf_attr, 0, -1, leader_fd, 0);
+  pfm_get_os_event_encoding("branch-misses", PFM_PLM3, PFM_OS_PERF_EVENT_EXT, &arg);
+  event_fd = perf_event_open(&perf_attr, 0, -1, leader_fd, 0);
+
+  struct read_format { uint64_t nr, time_enabled, time_running, values[4]; };
+  struct read_format before, after;

  for(j=0; j<ysz; j++) {
    for(i=0; i<xsz; i++) {
      double r = 0.0, g = 0.0, b = 0.0;
+     // capture counters before ray tracing
+     read(event_fd, &before, sizeof(struct read_format));

      for(s=0; s<samples; s++) {
        struct vec3 col = trace(get_primary_ray(i, j, s), 0);
        r += col.x;
        g += col.y;
        b += col.z;
      }
+     // capture counters after ray tracing
+     read(event_fd, &after, sizeof(struct read_format));

+     // save deltas in separate arrays
+     nanosecs[j * xsz + i] = after.time_running - before.time_running;
```

```
+     instrs  [j * xsz + i] = after.values[0] - before.values[0];
+     cycles  [j * xsz + i] = after.values[1] - before.values[1];
+     branches[j * xsz + i] = after.values[2] - before.values[2];
+     br_misps[j * xsz + i] = after.values[3] - before.values[3];

      *fb++ = ((uint32_t)(MIN(r * rcp_samples, 1.0) * 255.0) & 0xff) << RSHIFT |
              ((uint32_t)(MIN(g * rcp_samples, 1.0) * 255.0) & 0xff) << GSHIFT |
              ((uint32_t)(MIN(b * rcp_samples, 1.0) * 255.0) & 0xff) << BSHIFT;
  } }
+ // aggregate statistics and print it
  ...
}
```

在这个代码示例中，我们首先初始化libpfm库并配置性能事件以及我们将用于读取它们的格式。在 C-Ray 基准测试中，render函数只被调用一次。在您自己的代码中，务必小心不要多次进行libpfm初始化。然后，我们选择要分析的代码区域，在我们的案例中，它是一个带有trace函数调用的循环。我们用两个read系统调用包围这个代码区域，它们将在循环之前和之后捕获性能计数器的值。接下来，我们保存这些增量以供以后处理，例如，在这种情况下，我们通过计算平均值、90th 百分位数和最大值对其进行了聚合（代码未显示）。在基于 Intel Alderlake 的机器上运行它，我们得到了下面显示的输出。不需要 root 权限，但/proc/sys/kernel/perf_event_paranoid应该设置为小于 1。当在一个线程内读取计数器时，这些值仅适用于该线程。它可以选择性地包括运行并归因于该线程的内核代码。

```
$ ./c-ray-f -s 1024x768 -r 2 -i sphfract -o output.ppm
Per-pixel ray tracing stats:
                      avg         p90         max
-------------------------------------------------
nanoseconds   |      4571 |      6139 |     25567
instructions  |     71927 |     96172 |    165608
cycles        |     20474 |     27837 |    118921
branches      |      5283 |      7061 |     12149
branch-misses |        18 |        35 |       146
```

请记住，我们添加的插桩测量了每个像素的光线跟踪统计数据。将平均数乘以像素数（1024x768）应该给出大致的程序总统计数据。在这种情况下，一个很好的健全性检查是运行perf stat并比较我们收集的性能事件的整体 C-Ray 统计数据。

C-ray 基准测试主要强调 CPU 核心的浮点性能，通常不应该导致测量结果的高方差，换句话说，我们期望所有的测量结果都非常接近。然而，我们看到情况并非如此，因为 p90 值是平均值的 1.33 倍，而最大值有时比平均情况慢 5 倍。这里最可能的解释是对于一些像素，算法遇到了一个边界情况，执行了更多的指令，随后运行时间更长。但最好通过研究源代码或扩展插桩测量来捕获更多有关“慢”像素的数据，以确认假设。

Listing 64中显示的附加插桩测量代码导致了 17% 的开销，这对于本地实验来说是可以接受的，但在生产环境中运行的开销相当高。大多数大型分布式系统的目标是小于 1% 的开销，对于某些系统来说，最多可接受 5% 的开销，但是 17% 的减速不太可能让用户满意。管理插桩测量的开销至关重要，特别是如果您选择在生产环境中启用它。

开销通常以时间单位或工作单位（RPC、数据库查询、循环迭代等）的发生率来计算。如果我们系统上的一个系统调用大约需要 1.6 微秒的 CPU 时间，并且我们每个像素都执行两次（外部循环的迭代），那么每个像素的开销就是 3.2 微秒的 CPU 时间。

降低开销的策略有很多。作为一个通用原则，您的插桩测量应该始终具有固定的成本，例如，确定性系统调用，但

不是列表遍历或动态内存分配，否则它会干扰测量。插桩测量代码有三个逻辑部分：收集信息、存储信息和报告信息。为了降低第一部分（收集）的开销，我们可以减少采样率，例如，每 10 个 RPC 采样一次，然后跳过其余的。对于长时间运行的应用程序，性能可以通过相对便宜的随机采样进行监视 - 随机选择要观察的事件。这些方法牺牲了收集的准确性，但仍然提供了对整体性能特征的良好估计，同时产生了非常低的开销。

对于第二部分和第三部分（存储和聚合），建议仅收集、处理和保留您需要了解系统性能的数据量。您可以通过使用“在线”算法来计算平均值、方差、最小值、最大值和其他指标来避免将每个样本存储在内存中。这将大大减少插桩测量的内存占用。例如，方差和标准差可以使用 Knuth 的在线方差算法来计算。一个良好的实现[65]使用不到 50 字节的内存。

对于长时间运行的例程，您可以在开始、结束和一些中间部分收集计数器。在连续运行中，您可以二分搜索执行最差的例程部分并进行优化。重复此过程，直到所有性能差的地方都被消除。如果尾延迟是主要关注的问题，那么在特别慢的运行中发出日志消息可以提供有用的见解。

在 Listing 64中，我们同时收集了 4 个事件，尽管 CPU 有 6 个可编程计数器。您可以打开具有不同事件集的其他组。内核将选择不同的组来运行。`time_enabled`和`time_running`字段指示了多路复用。它们都是以纳秒为单位的持续时间。`time_enabled`字段表示事件组已启用的纳秒数。`time_running`表示实际收集事件的时间占已启用时间的多少。如果同时启用了两个无法放在 HW 计数器上的事件组，您可能会看到它们都收敛到`time_running = 0.5 * time_enabled`。调度通常很复杂，因此在依赖于您的确切场景之前，请进行验证。

同时捕获多个事件允许计算我们在第 4 章中讨论的各种指标。例如，捕获 `INSTRUCTIONS_RETIRED` 和 `UNHALTED_CLOCK_CYCLES` 使我们能够测量 IPC。我们可以通过比较 CPU 周期（`UNHALTED_CORE_CYCLES`）和固定频率参考时钟（`UNHALTED_REFERENCE_CYCLES`）来观察频率缩放的影响。通过请求消耗的 CPU 周期（`UNHALTED_CORE_CYCLES`，仅在线程运行时计数）并与墙钟时间进行比较，可以检测线程未运行的情况。此外，我们可以对数字进行归一化，以获得每秒/时钟/指令的事件速率。例如，通过测量 `MEM_LOAD_RETIRED.L3_MISS` 和 `INSTRUCTIONS_RETIRED`，我们可以获得`L3MPKI`指标。正如您所见，这种设置非常灵活。

事件分组的重要属性是计数器将原子地在同一次`read`系统调用下可用。这些原子束非常有用。首先，它允许我们在每个组内相关事件。例如，我们为代码区域测量 IPC，并发现它非常低。在这种情况下，我们可以将两个事件（指令和周期）与第三个事件配对，例如 L3 缓存丢失，以检查它是否对我们正在处理的低 IPC 有贡献。如果没有，我们将继续使用其他事件进行因子分析。其次，事件分组有助于减轻工作负载具有不同阶段的偏差。由于组内的所有事件同时测量，它们始终捕获相同的阶段。

在某些场景中，插桩测量可能成为功能或特性的一部分。例如，开发人员可以实现一个插桩测量逻辑，用于检测 IPC 的下降（例如，当有一个繁忙的兄弟硬件线程运行时）或 CPU 频率的下降（例如，由于负载过重而导致系统节流）。当发生这种事件时，应用程序会自动推迟低优先级的工作以补偿临时增加的负载。

5.5 采样

采样是最常用的性能分析方法。人们通常将其与程序中的热点识别联系起来。广义而言之，采样有助于找到代码中对特定性能事件贡献最多的位置。如果我们考虑发现热点，那么这个问题可以重新表述为程序中的哪个地方消耗了最多的 CPU 周期。人们通常将技术上称为采样的操作称为“性能分析”。根据维基百科https://en.wikipedia.org/wiki/Profiling_(computer_programming)[66]的说法，性能分析是一个更广泛的术语，包括各种收集数据的技术，例如中断、代码检测和 PMC。

令人惊讶的是，人们可以想象到的最简单的采样性能分析器就是调试器。事实上，您可以通过以下步骤识别热点：a) 在调试器下运行程序，b) 每 10 秒暂停一次程序，c) 记录程序停止的位置。如果您多次重复 b) 和 c)，您就可以从这些样本构建一个直方图。您停止最多的代码行将是程序中最热的代码行。当然，这不是一种高效的发现热点的方

[65] 准确计算运行方差 - https://www.johndcook.com/blog/standard_deviation/

[66] Profiling(wikipedia) - https://en.wikipedia.org/wiki/Profiling_(computer_programming.

法，我们也不推荐这样做。它只是为了说明这个概念。尽管如此，它是关于真实性能分析工具如何工作的简化描述。现代性能分析器每秒可以收集数千个样本，这为基准测试中的热点提供了相当准确的估计。

与调试器的例子一样，每次捕获新的样本时，被分析程序的执行都会中断。在中断时，性能分析器会收集程序状态的快照，构成一个样本。为每个样本收集的信息可能包括中断时执行的指令地址、寄存器状态、调用堆栈（见 Section 5.5.3），等等。收集到的样本存储在一个转储文件中，该文件可以进一步用于显示程序中耗时最多的部分、调用图等。

5.5.1 用户模式和基于硬件事件的采样

采样可以采用两种不同的模式进行，即用户模式采样或基于硬件事件的采样 (EBS)。用户模式采样是一种纯软件方法，将代理库嵌入到被分析的应用程序中。代理为应用程序中的每个线程设置操作系统计时器。计时器到期后，应用程序会收到由收集器处理的 `SIGPROF` 信号。EBS 使用硬件 PMC 触发中断。特别是，它使用 PMU 的计数器溢出功能，我们将在稍后讨论。

用户模式采样只能用于识别热点，而 EBS 可用于涉及 PMC 的其他分析类型，例如，基于缓存未命中、TMA（见 Section 6.1）等进行采样。

与 EBS 相比，用户模式采样的运行时开销更大。当采样间隔为 10ms 时，用户模式采样的平均开销约为 5%，而 EBS 的开销不到 1%。由于开销更低，您可以使用更高的采样率使用 EBS，这将提供更准确的数据。然而，用户模式采样生成的数据更少，因此处理起来也更快。

5.5.2 寻找热点

在本节中，我们将讨论使用 PMC 和 EBS 的机制。图 31 说明了 PMU 的计数器溢出功能，该功能用于触发性能监控中断 (PMI)，也称为 `SIGPROF`。在基准测试开始时，我们会配置我们想要采样的事件。识别热点意味着知道程序花费大部分时间在哪里。因此，在周期上进行采样是非常自然的，这也是许多性能分析工具的默认设置。但这并不一定是严格的规则；我们可以对任何想要的性能事件进行采样。例如，如果我们想知道程序中 L3 缓存未命中最多的位置，我们将在相应的事件上进行采样，即 `MEM_LOAD_RETIRED.L3_MISS`。

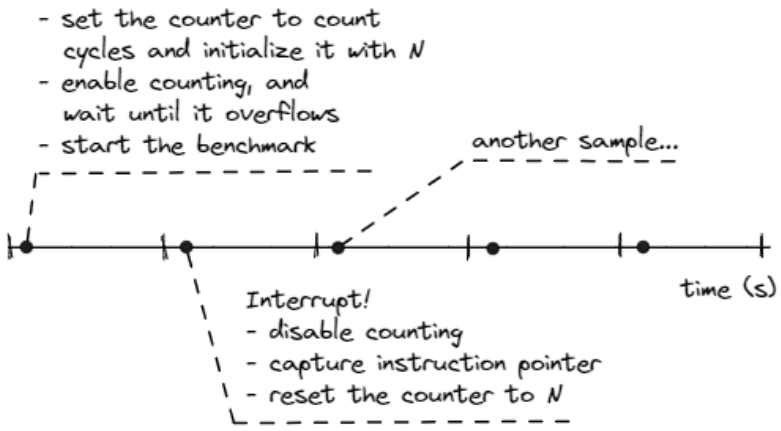


Figure 31: 使用性能计数器进行采样

初始化寄存器后，我们开始计数并让基准测试继续。我们将 PMC 配置为计数周期，因此它将在每个周期递增。最终，它会溢出。当寄存器溢出时，硬件将引发 PMI。性能分析工具被配置为捕获 PMI，并具有用于处理它们的中断服务程序 (ISR)。我们在 ISR 中执行多个步骤：首先，我们禁用计数；然后，我们记录 CPU 在计数器溢出时执行的指令；然后，我们将计数器重置为 `N` 并恢复基准测试。

现在，让我们回到值 N。使用这个值，我们可以控制我们想要多久获得一个新的中断。假设我们想要更细粒度的粒度，每 100 万条指令获得一个样本。为了实现这一点，我们可以将计数器设置为 (unsigned) -1'000'000，这样它将在每 100 万条指令后溢出。这个值也称为“采样后”值。

我们重复这个过程多次，以建立足够的样本集合。如果我们稍后聚合这些样本，就可以构建程序中最热位置的直方图，就像下面 Linux perf record/report 输出所示。这给了我们按降序排序的程序函数开销的细分（热点）。下面展示了一个来自 Phoronix 测试套件: https://www.phoronix-test-suite.com/[67] 的 x264: https://openbenchmarking.org/test/pts/x264[68] 基准测试的采样示例：

```
$ time -p perf record -F 1000 -- ./x264 -o /dev/null --slow --threads 1
    ../Bosphorus_1920x1080_120fps_420_8bit_YUV.y4m
[ perf record: Captured and wrote 1.625 MB perf.data (35035 samples) ]
real 36.20 sec
$ perf report -n --stdio
# Samples: 35K of event 'cpu_core/cycles/'
# Event count (approx.): 156756064947
# Overhead  Samples  Shared Object  Symbol
# ........  .......  .............  ....................................
  7.50%     2620     x264           [.] x264_8_me_search_ref
  7.38%     2577     x264           [.] refine_subpel.lto_priv.0
  6.51%     2281     x264           [.] x264_8_pixel_satd_8x8_internal_avx2
  6.29%     2212     x264           [.] get_ref_avx2.lto_priv.0
  5.07%     1787     x264           [.] x264_8_pixel_avg2_w16_sse2
  3.26%     1145     x264           [.] x264_8_mc_chroma_avx2
  2.88%     1013     x264           [.] x264_8_pixel_satd_16x8_internal_avx2
  2.87%     1006     x264           [.] x264_8_pixel_avg2_w8_mmx2
  2.58%      904     x264           [.] x264_8_pixel_satd_8x8_avx2
  2.51%      882     x264           [.] x264_8_pixel_sad_16x16_sse2
  ...
```

Linux perf 采集了 35'035 个样本，这意味着中断执行的过程发生了这么多次。我们还使用了 -F 1000，将采样率设置为每秒 1000 个样本。这大致与 36.2 秒的整体运行时间相匹配。请注意，Linux perf 提供了大约经过的总周期数。如果我们将它除以样本数，我们将得到 156756064947 个周期 / 35035 个样本 = 450 万个周期/样本。这意味着 Linux perf 将数字 N 设置为大约 4'500'000 以每秒收集 1000 个样本。数字 N 可以由工具根据实际 CPU 频率动态调整。

当然，对我们最有价值的是按每个函数分配的样本数量排序的热点列表。在知道最热门的函数之后，我们可能想要更深入地研究：每个函数内部代码的热门部分是什么。要查看内联函数的优化分析文件数据以及为特定源代码区域生成的汇编代码，我们需要使用调试信息（-g 编译器标志）构建应用程序。

调试信息有两个主要用例：调试功能问题（错误）和性能分析。对于功能调试，我们需要尽可能多的信息，这是您传递 -g 编译器标志时的默认设置。但是，如果用户不需要完整的调试体验，那么只需要行号就足以进行性能分析。您可以通过使用 -gline-tables-only 选项将生成的调试信息量减少到代码中符号的行号。[69]

Linux perf 没有丰富的图形支持，因此查看源代码的热门部分非常不方便，但可以做到。Linux perf 将源代码与生

[67] Phoronix test suite - https://www.phoronix-test-suite.com/.

[68] x264 benchmark - https://openbenchmarking.org/test/pts/x264.

[69] 过去，使用调试信息 (-g) 编译时存在 LLVM 编译器错误。代码转换传递错误地处理了调试内部函数的存在，导致了不同的优化决策。它不会影响功能，只会影响性能。其中一些已经修复，但很难说是否存在其他问题。

成的汇编代码混合在一起，如下所示：

```
# snippet of annotating source code of 'x264_8_me_search_ref' function
$ perf annotate x264_8_me_search_ref --stdio
Percent | Source code & Disassembly of x264 for cycles:ppp
----------------------------------------------------------
  ...
       :                 bmx += square1[bcost&15][0];   <== source code
  1.43 : 4eb10d:  movsx  ecx,BYTE PTR [r8+rdx*2]        <== corresponding machine code
       :                 bmy += square1[bcost&15][1];
  0.36 : 4eb112:  movsx  r12d,BYTE PTR [r8+rdx*2+0x1]
       :                 bmx += square1[bcost&15][0];
  0.63 : 4eb118:  add    DWORD PTR [rsp+0x38],ecx
       :                 bmy += square1[bcost&15][1];
  ...
```

大多数带有图形用户界面 (GUI) 的性能分析器，例如 Intel VTune Profiler，都可以并排显示源代码和关联的汇编代码。此外，还有一些工具可以以类似于 Intel Vtune 和其他工具的丰富图形界面可视化 Linux `perf` 原始数据的输出。您将在第 7 章中更详细地看到所有这些内容。

5.5.3 收集调用堆栈

在采样时，我们经常会遇到程序中最热门的函数被多个函数调用的情况。图 32 显示了一个这样的场景示例。性能分析工具的输出可能显示 `foo` 是程序中最热门的函数之一，但如果它有多个调用者，我们想知道哪个调用者调用 `foo` 的次数最多。对于程序中出现诸如 `memcpy` 或 `sqrt` 之类的库函数的热点，这是典型情况。要了解特定的函数为什么成为热点，我们需要知道程序控制流图 (CFG) 中哪个路径导致了这种情况。

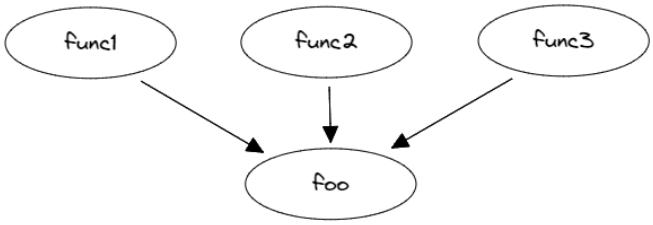


Figure 32: 控制流图: 热点函数 “foo” 有多个调用者

分析 `foo` 所有调用者的源代码可能非常耗时。我们只想关注那些导致 `foo` 成为热点的调用者。换句话说，我们想要找出程序 CFG 中最热门的路径。性能分析工具通过在收集性能样本时捕获进程的调用堆栈和其他信息来实现这一点。然后，对所有收集到的堆栈进行分组，使我们能够看到导致特定函数的最热门路径。

在 Linux `perf` 中，可以使用三种方法收集调用堆栈：

1. **帧指针 (`perf record --call-graph fp`)**。要求使用 `--fnoomit-frame-pointer` 构建二进制文件。历史上，帧指针（RBP 寄存器）用于调试，因为它使我们能够在不弹出所有参数的情况下获取调用堆栈（也称为堆栈展开）。帧指针可以立即告诉返回地址。但是，它仅为此目的占用了一个寄存器，所以开销很大。它也可用用于性能分析，因为它可以进行廉价的堆栈展开。
2. **DWARF 调试信息(`perf record --call-graph dwarf`)**。要求使用 DWARF 调试信息 `-g(-gline-tables-only)` 构建二进制文件。通过堆栈展开过程获取调用堆栈。

3. **英特尔最后分支记录 (LBR) 硬件功能 (`perf record --call-graph lbr`)**。通过解析 LBR 堆栈（一组硬件寄存器）获取调用堆栈。调用图不像前两种方法那么深。有关 LBR 的更多信息，请参见 Section 6.2。

下面是使用 LBR 在程序中收集调用堆栈的示例。通过查看输出，我们知道 55% 的时间 `foo` 是由 `func1` 调用的，33% 的时间是由 `func2` 调用的，11% 的时间是由 `fun3` 调用的。我们可以清楚地看到 `foo` 的调用者之间的开销分布，现在可以将注意力集中在程序 CFG 中最热的边 `func1 -> foo` 上，但我们也应该关注边 `func2 -> foo`。

```
$ perf record --call-graph lbr -- ./a.out
$ perf report -n --stdio --no-children
# Samples: 65K of event 'cycles:ppp'
# Event count (approx.): 61363317007
# Overhead       Samples  Command  Shared Object     Symbol
# ........  ............  .......  ................  ......................
    99.96%         65217  a.out    a.out             [.] foo
           |
            --99.96%--foo
                      |
                      |--55.52%--func1
                      |          main
                      |          __libc_start_main
                      |          _start
                      |
                      |--33.32%--func2
                      |          main
                      |          __libc_start_main
                      |          _start
                      |
                       --11.12%--func3
                                 main
                                 __libc_start_main
                                 _start
```

当使用 Intel VTune Profiler 时，可以在配置分析时勾选相应的“收集堆栈”框来收集调用堆栈数据。当使用命令行界面时，指定 `-knob enable-stack-collection=true` 选项。

知道一种有效的收集调用堆栈的方法非常重要。不熟悉该概念的开发人员会尝试使用调试器来获取此信息。他们通过中断程序的执行并分析调用堆栈（例如，gdb 调试器中的 `backtrace` 命令）来做到这一点。不要这样做，让性能分析工具来完成这项工作，它更快、更准确。

5.6 Roofline 性能模型

Roofline 性能模型是一个以吞吐量为导向的性能模型，在 HPC 领域广泛使用。它于 2009 年在加州大学伯克利分校开发。模型中的“roofline”表示应用程序的性能不能超过机器的能力。程序中的每个函数和每个循环都受到机器的计算或内存容量的限制。这个概念在图 33 中有所体现。应用程序的性能始终会受到某条“roofline”函数的限制。

硬件有两个主要限制：计算速度 (峰值计算性能，FLOPS) 和数据移动速度 (峰值内存带宽，GB/s)。应用程序的最大性能受峰值 FLOPS（水平线）和平台带宽乘以运算强度（对角线）两者之间的最小值限制。图 33 中的 roofline 图将两个应用程序 `A` 和 `B` 的性能与硬件限制进行了对比。应用程序 `A` 的运算强度较低，其性能受内存带宽限制，而应用

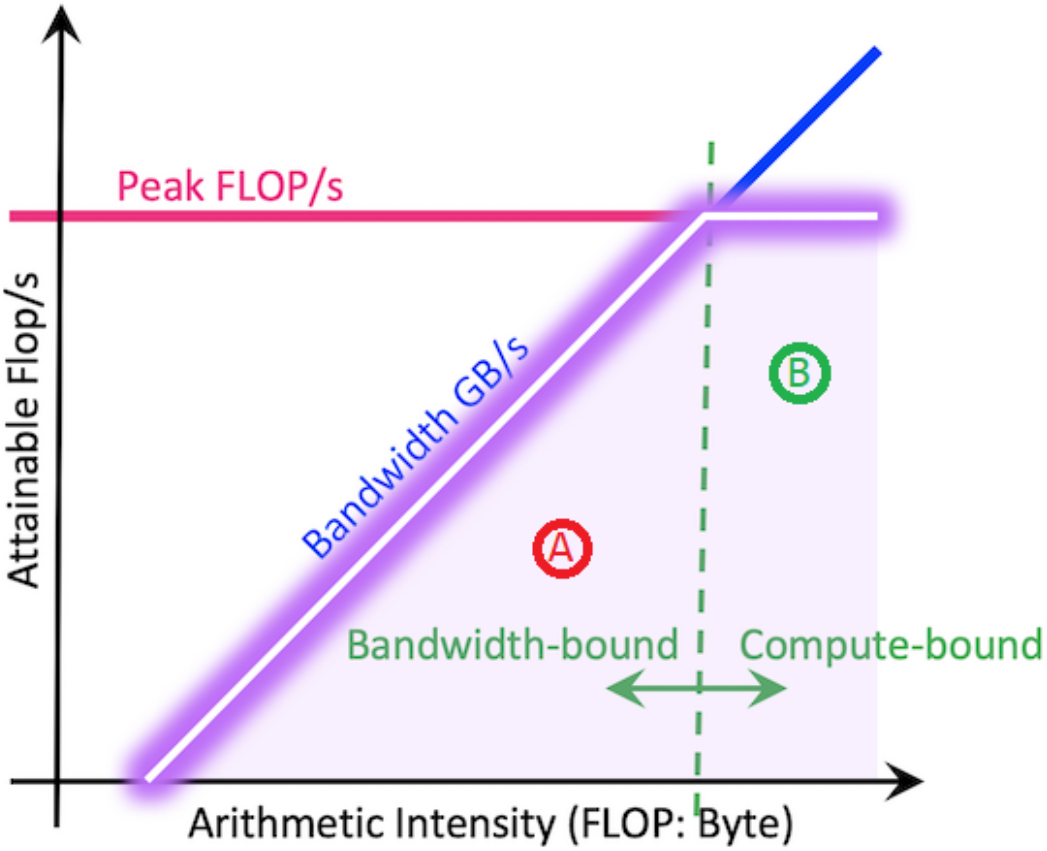


Figure 33: Roofline model. © Image taken from NERSC Documentation.

程序 B 的计算密集型程度更高，因此不会受到内存瓶颈的太大影响。类似地，A 和 B 可以代表程序中的两个不同函数，并具有不同的性能特征。Roofline 性能模型会考虑到这一点，可以在同一个图表上显示应用程序的多个函数和循环。

算术强度 (Arithmetic Intensity, AI) 是 FLOPS 和字节之间的比率，可以针对程序中的每个循环进行提取。让我们计算 Listing 5.6 中代码的算术强度。在最内层的循环体中，我们有一个加法和一个乘法；因此，我们有 2 个 FLOP。此外，我们还有三个读取操作和一个写入操作；因此，我们传输了 `4 ops * 4 bytes = 16` 个字节。该代码的算术强度为 `2 / 16 = 0.125`。AI 是给定性能点的 X 轴上的值。

清单：朴素并行矩阵乘法。

```
void matmul(int N, float a[][2048], float b[][2048], float c[][2048]) {
    #pragma omp parallel for
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                c[i][j] = c[i][j] + a[i][k] * b[k][j];
            }
        }
    }
}
```

传统的应用程序性能提升方式是充分利用机器的 SIMD 和多核能力。通常情况下，我们需要优化多个方面：向量化、内存、线程。Roofline 方法可以帮助评估应用程序的这些特性。在 roofline 图表上，我们可以绘制标量单核、SIMD 单核和 SIMD 多核性能的理论最大值（见图 34 ）。这将使我们了解改进应用程序性能的空间。如果我们发现我们的应用程序受计算绑定（即具有高算术强度）并且低于峰值标量单核性能，我们应该考虑强制向量化（参见 Section 9.3.5）并将工作分发到多个线程上。相反，如果应用程序的算术强度低，我们应该寻求改善内存访问的方法（参见 Chapter 8）。使用 Roofline 模型优化性能的最终目标是向上移动这些点。向量化和线程化向上移动点，而通过增加算术强度优化内存访问则会将点向右移动，并且可能也会提高性能。

理论最大值（roofline）通常在设备规范中给出，可以很容易地查阅。您也可以根据您使用的机器的特性计算理论最大值。一旦您知道机器的参数，这通常并不难做。对于 Intel Core i5-8259U 处理器，使用 AVX2 和 2 个 Fused Multiply Add (FMA) 单元的最大 FLOP 数（单精度浮点）可以计算如下：

$$\begin{aligned}\text{峰值 FLOPS} = 8\,(\text{逻辑核心数量}) \times \frac{256\,(\text{AVX 位宽})}{32\,\text{位 (float 大小)}} \times \\ 2\,(\text{FMA}) \times\ 3.8\ \text{GHz}\,(\text{最大睿频}) \\ = 486.4\ \text{GFLOPs}\end{aligned}$$

我用于实验的 Intel NUC Kit NUC8i5BEH 的最大内存带宽可以如下计算。请记住，DDR 技术允许每次内存访问传输 64 位或 8 个字节。

$$\begin{aligned}\text{峰值内存带宽} = 2400\,(\text{DDR4 内存传输速率}) \times\ 2\,(\text{内存通道}) \times \\ 8\,(\text{每次内存访问的字节数}) \times 1\,(\text{插槽}) = 38.4\ \text{GiB/s}\end{aligned}$$

像 Empirical Roofline Tool: https://bitbucket.org/berkeleylab/cs-roofline-toolkit/src/master/[70] 和 Intel Advisor:

[70] Empirical Roofline Tool - https://bitbucket.org/berkeleylab/cs-roofline-toolkit/src/master/.

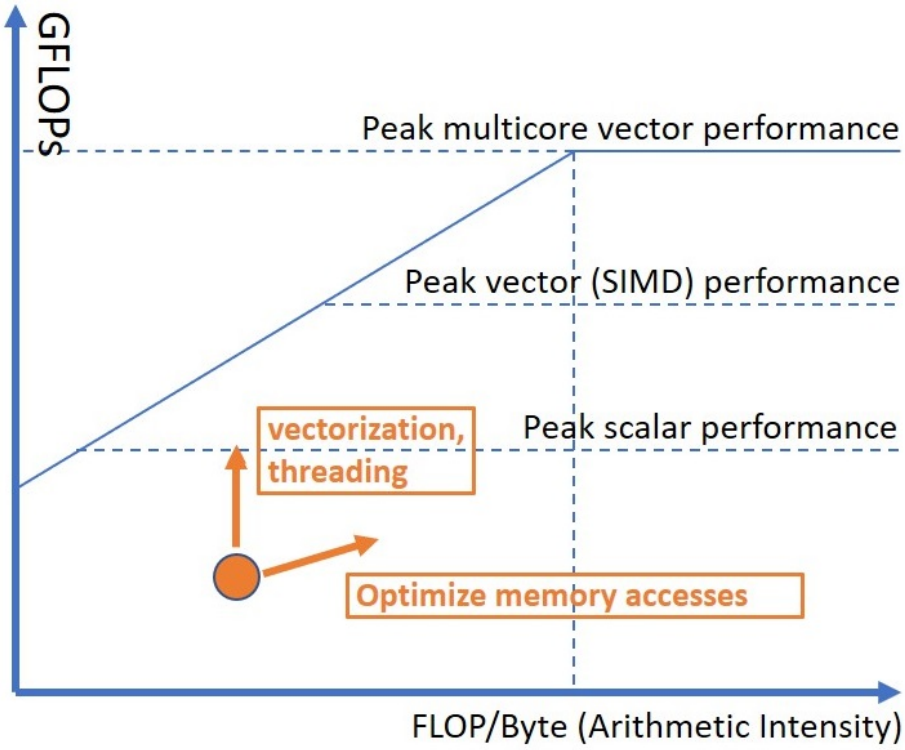


Figure 34: Roofline 性能模型

https://software.intel.com/content/www/us/en/develop/tools/advisor.html[71] 这样的自动化工具能够通过运行一组预先准备的基准测试来经验性地确定理论最大值。如果一个计算可以重用缓存中的数据，则可以实现更高的 FLOP 速度。Roofline 可以通过为每个内存层次引入专门的 roofline 来实现这一点（参见图 35 ）。

确定硬件限制后，我们可以开始评估应用程序相对于 roofline 的性能。用于自动收集 Roofline 数据的两种最常用方法是采样（由 likwid: https://github.com/RRZE-HPC/likwid[72] 工具使用）和二进制插桩（由 Intel 软件开发仿真器 (SDE: https://software.intel.com/content/www/us/en/develop/articles/intel-software-development-emulator.html[73]) 使用）。采样在数据收集方面产生的开销较低，而二进制插桩则能提供更准确的结果。[74] Intel Advisor 自动构建 Roofline 图表，并为给定循环的性能优化提供提示。图 35 展示了 Intel Advisor 生成的 Roofline 图表示例。请注意，Roofline 图表使用的是对数刻度。

Roofline 方法可以通过在同一个图表上打印“之前”和“之后”的点来跟踪优化进度。因此，它是一个迭代的过程，指导开发人员帮助他们的应用程序充分利用硬件功能。图 35 显示了对之前在 Listing 5.6 中显示的代码进行以下两个更改所带来的性能提升：

- 交换两个最内层的循环（交换第 4 和第 5 行）。这可以实现缓存友好的内存访问（参见 Chapter 8）。
- 使用 AVX2 指令启用最内层循环的自动向量化。

总结来说，Roofline 性能模型可以帮助：

- 识别性能瓶颈。
- 指导软件优化。
- 确定优化何时结束。

[71] Intel Advisor - https://software.intel.com/content/www/us/en/develop/tools/advisor.html.

[72] Likwid - https://github.com/RRZE-HPC/likwid.

[73] Intel SDE - https://software.intel.com/content/www/us/en/develop/articles/intel-software-development-emulator.html.

[74] 在此演示文稿中，可以看到更详细的收集 roofline 数据的方法比较：https://crd.lbl.gov/assets/Uploads/ECP20-Roofline-4-cpu.pdf

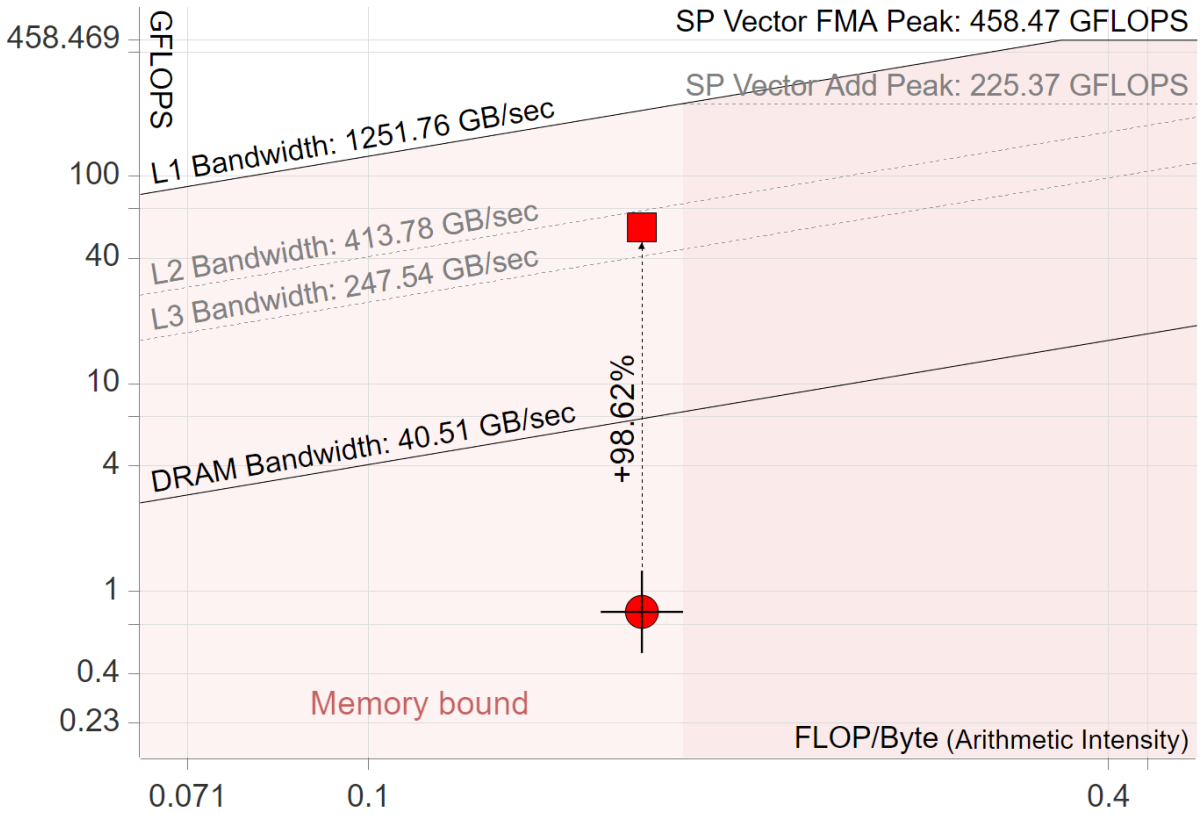


Figure 35: 在 Intel NUC 工具包 NUC8i5BEH 上使用 clang 10 编译器对矩阵乘法进行了 Roofline 分析

- 相对于机器能力评估性能。

其他资源和链接:

- NERSC 文档，网址：https://docs.nersc.gov/development/performance-debugging-tools/roofline/。
- 劳伦斯伯克利国家实验室研究，网址：https://crd.lbl.gov/departments/computer-science/par/research/roofline/
- 关于 Roofline 模型和 Intel Advisor 的视频演示集合，网址：https://techdecoded.intel.io/ (搜索 “Roofline”)。
- `Perfplot` 是一个脚本和工具集合，允许用户在最近的 Intel 平台上测量性能计数器，并使用结果生成 roofline 和性能图。网址：https://github.com/GeorgOfenbeck/perfplot

5.7 静态性能分析

如今，我们拥有广泛的静态代码分析工具。对于 C 和 C++ 语言，我们有一些著名的工具，例如 Clang 静态分析器: https://clang-analyzer.llvm.org/、Klocwork: https://www.perforce.com/products/klocwork、Cppcheck: http://cppcheck.sourceforge.net/ 等。它们旨在检查代码的正确性和语义。同样，也有一些工具试图解决代码的性能方面的问题。静态性能分析器不会执行或分析程序，而是模拟代码，就好像它在真实硬件上执行一样。静态预测性能几乎是不可能的，因此这种类型的分析有很多限制。

首先，由于我们不知道要编译成的机器代码，所以不可能静态分析 C/C++ 代码的性能。因此，静态性能分析针对的是汇编代码。

其次，静态分析工具模拟工作负载而不是执行它。这显然非常慢，因此不可能静态分析整个程序。相反，工具会取一小段汇编代码，并试图预测它在真实硬件上的行为。用户应该选择特定的汇编指令（通常是小型循环）进行分析。因此，静态性能分析的范围非常窄。

静态性能分析器的输出相当低级，有时会将执行分解到 CPU 周期。通常，开发人员将其用于关键代码区域的细粒度

调整，其中每个 CPU 周期都很重要。

静态分析器 vs. 动态分析器

静态工具: 不运行实际代码，而是尝试模拟执行，尽可能保留微架构细节。它们无法进行实际测量（执行时间、性能计数器），因为它们不运行代码。优点是您不需要拥有真正的硬件，可以针对不同代的 CPU 模拟代码。另一个好处是您不必担心结果的一致性：静态分析器总是会给您确定性的输出，因为模拟（与实际硬件上的执行相比）不会出现任何偏差。静态工具的缺点是它们通常无法预测和模拟现代 CPU 中的所有内容：它们基于一个可能存在错误和限制的模型。静态性能分析器的例子包括 UICA: https://uica.uops.info/[75] 和 llvm-mca: https://llvm.org/docs/CommandGuide/llvm-mca.html[76].

动态工具: 基于在真实硬件上运行代码并收集有关执行的所有信息。这是证明任何性能假设的唯一 100% 可靠的方法。缺点是，通常您需要具有特权访问权限才能收集低级性能数据，例如 PMCs。编写一个好的基准测试并测量您想要测量的内容并不总是容易的。最后，您需要过滤噪音和不同类型的副作用。动态微架构性能分析器的例子包括 nanoBench: https://github.com/andreas-abel/nanoBench,[77] uarch-bench: https://github.com/travisdowns/uarch-bench[78] 等。

一个更大的静态和动态微架构性能分析工具集合可以在 这里: https://github.com/MattPD/cpplinks/blob/master/performance.tools.md#microarchitecture[79] 获得。

5.7.1 案例研究：使用 UICA 优化 FMA 吞吐量

开发人员经常会问的一个问题是："最新处理器拥有 10 多个执行单元；我该如何编写代码让它们一直保持繁忙？"这确实是一个最难解决的问题之一。有时它需要仔细观察程序如何运行。UICA 模拟器就是这样一个显微镜，可以让您深入了解您的代码如何流经现代处理器。

让我们看一下 Listing 5.7.1 中的代码。我们有意使示例尽可能简单。当然，现实世界中的代码通常比这更复杂。该代码将数组 `a` 的每个元素乘以常数 `B`，并将缩放后的值累积到 `sum` 中。在右侧，我们展示了使用 `-O3 -ffast-math -march=core-avx2` 编译时 Clang-16 生成的循环的机器代码。汇编代码看起来非常紧凑，让我们更好地理解它。

清单：FMA 吞吐量

```
float foo(float * a, float B, int N){
  float sum = 0;
  for (int i = 0; i < N; i++)
    sum += a[i] * B;
  return sum;
}
```

```
.loop:
 vfmadd231ps ymm2, ymm1, ymmword [rdi + rsi]
 vfmadd231ps ymm3, ymm1, ymmword [rdi + rsi + 32]
 vfmadd231ps ymm4, ymm1, ymmword [rdi + rsi + 64]
 vfmadd231ps ymm5, ymm1, ymmword [rdi + rsi + 96]
 sub rsi, -128
 cmp rdx, rsi
 jne .loop
```

这段代码是一个归约循环，即我们需要对所有乘积求和，最终返回一个浮点数。按照目前代码的写法，`sum` 上存在循环传递依赖性。您无法覆盖 `sum`，直到累积上一个乘积。一种并行化的巧妙方法是使用多个累加器并在最后将它们汇总。因此，我们可以用多个累加器代替单个 `sum`，例如 `sum1` 用于累积偶数次迭代的结果，`sum2` 用于累积奇数次迭代的结果。这就是 Clang-16 所做的：它使用了 4 个向量寄存器（`ymm2-ymm5`），每个都包含 8 个浮点累加器，并

[75] UICA - https://uica.uops.info/

[76] LLVM MCA - https://llvm.org/docs/CommandGuide/llvm-mca.html

[77] nanoBench - https://github.com/andreas-abel/nanoBench

[78] uarch-bench - https://github.com/travisdowns/uarch-bench

[79] C++ 性能工具链接集合 - https://github.com/MattPD/cpplinks/blob/master/performance.tools.md#microarchitecture.

使用 FMA 将乘法和加法融合成单个指令。常量 B 被广播到 ymm1 寄存器中。-ffast-math 选项允许编译器重新关联浮点运算，我们将在 Section 9.3.5 中讨论这个选项如何帮助优化。顺便说一句，乘法在循环后只需要做一次。这肯定是程序员的疏忽，但希望编译器将来能够处理它。

代码看起来不错，但它真的是最优的吗？让我们找出答案。我们将 Listing 5.7.1 中的汇编代码片段带到 UICA 进行模拟。在撰写本文时，UICA 不支持 Alderlake (英特尔第 12 代，基于 GoldenCove)，因此我们在最新可用的 RocketLake (英特尔第 11 代，基于 SunnyCove) 上运行了它。虽然架构不同，但这次实验暴露的问题在两者上都同样明显。模拟结果如图 36 所示。这是一个类似于我们在第 3 章中展示的流水线图。我们跳过了前两个迭代，只显示了第 2 和第 3 个迭代（最左列 “It.”）。这时，执行已经达到稳定状态，所有后续迭代看起来都非常相似。

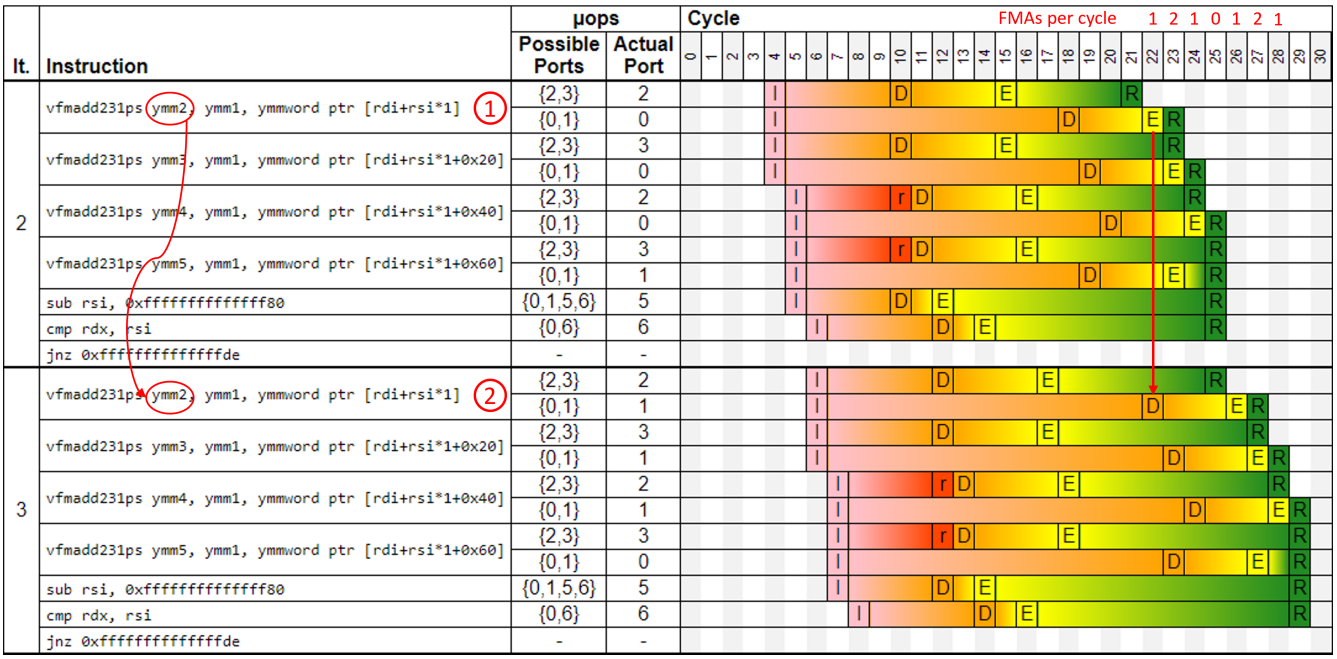


Figure 36: UICA pipeline diagram. I = issued, r = ready for dispatch, D = dispatched, E = executed, R = retired.

UICA 是一个非常简化的实际 CPU 流水线模型。例如，您可能会注意到指令提取和译码阶段丢失了。此外，UICA 不考虑缓存未命中和分支预测错误，因此它假设所有内存访问总是命中 L1 缓存并且分支总是预测正确。我们都知道这在现代处理器中并非如此。同样，这与我们的实验无关，因为我们仍然可以使用模拟结果来找到改进代码的方法。您能看到问题吗？

让我们仔细看看这个图表。首先，每个 FMA 指令都被分解成两个微操作 (见 ①)：一个加载微操作，进入端口 {2,3}，一个 FMA 微操作，可以进入端口 {0,1}。加载微操作的延迟是 5 个周期：从第 11 个周期开始到第 15 个周期结束。FMA 微操作的延迟是 4 个周期：从第 19 个周期开始到第 22 个周期结束。所有 FMA 微操作都依赖于加载微操作，我们可以在图表上清楚地看到这一点：FMA 微操作总是对应加载微操作完成后才开始。现在找到第 10 个周期中的两个 r 单元格，它们已经准备调度，但是 RocketLake 只有两个加载端口，并且在同一个周期都被占用了。因此，这两个加载指令在下个周期发出。

该循环在 ymm2-ymm5 上具有四个跨迭代依赖性。来自指令 ② 的写入 ymm2 的 FMA 微操作无法在上一迭代的指令 ① 完成之前开始执行。请注意，来自指令 ② 的 FMA 微操作与指令 ① 完成执行的同一个周期 22 被调度。您也可以观察其他 FMA 指令的这种模式。

那么，您可能会问，“问题是什么？”。请看图片的右上角。对于每个周期，我们都计算了已执行的 FMA 微操作的数量，这不是 UICA 打印的。它看起来像 1,2,1,0,1,2,1,...，或者平均每个周期 1 个 FMA 微操作。最近的英特尔处理器大多有两个 FMA 执行单元，因此每个周期可以发出两个 FMA 微操作。该图表清楚地显示了差距，因为每个第

四个周期都没有执行 FMA 指令。正如我们之前发现的，由于它们的输入 (ymm2-ymm5) 没有准备好，因此无法调度任何 FMA 微操作。

为了将 FMA 执行单元的利用率从 50% 提高到 100%，我们需要将循环展开两倍。这将使累加器的数量从 4 个增加到 8 个。此外，我们将有 8 个独立的数据流链，而不是 4 个。我们这里不会展示展开版本的模拟，您可以自己尝试。相反，让我们通过在真实硬件上运行两个版本来确认假设。顺便说一句，这是一个好主意，因为 UICA 等静态性能分析器并不是准确的模型。下面，我们展示了我们在最近的 Alderlake 处理器上运行的两个 nanobench: https://github.com/andreas-abel/nanoBench 测试的输出。该工具采用提供的汇编指令 (-asm 选项) 并创建一个 benchmark 内核。读者可以查阅 nanobench 文档中其他参数的含义。左侧的原始代码在 4 个周期内执行 4 条指令，而改进后的版本可以在 4 个周期内执行 8 条指令。现在我们可以确定我们最大化了 FMA 执行吞吐量，右侧的代码使 FMA 单元始终处于忙碌状态。

```
# ran on Intel Core i7-1260P (Alderlake)
$ sudo ./kernel-nanoBench.sh -f -unroll 10
 -loop 100 -basic -warm_up_count 10 -asm "
VFMADD231PS YMM0, YMM1, ymmword [R14];
VFMADD231PS YMM2, YMM1, ymmword [R14+32];
VFMADD231PS YMM3, YMM1, ymmword [R14+64];
VFMADD231PS YMM4, YMM1, ymmword [R14+96];"
-asm_init "<not shown>"

Instructions retired: 4.20
Core cycles: 4.02
```

```
$ sudo ./kernel-nanoBench.sh -f -unroll 10
 -loop 100 -basic -warm_up_count 10 -asm "
VFMADD231PS YMM0, YMM1, ymmword [R14];
VFMADD231PS YMM2, YMM1, ymmword [R14+32];
VFMADD231PS YMM3, YMM1, ymmword [R14+64];
VFMADD231PS YMM4, YMM1, ymmword [R14+96];
VFMADD231PS YMM5, YMM1, ymmword [R14+128];
VFMADD231PS YMM6, YMM1, ymmword [R14+160];
VFMADD231PS YMM7, YMM1, ymmword [R14+192];
VFMADD231PS YMM8, YMM1, ymmword [R14+224]"
-asm_init "<not shown>"

Instructions retired: 8.20
Core cycles: 4.02
```

作为经验法则，在这种情况下，循环必须按 `T * L` 的倍数展开，其中 `T` 是指令的吞吐量，`L` 是其延迟。在我们的案例中，由于 Alderlake 上 FMA 的吞吐量为 2，延迟为 4 个周期，因此我们应该将其展开 `2 * 4 = 8` 倍以实现最大 FMA 端口利用率。这会创建 8 个可以独立执行的单独数据流链。

值得一提的是，在实践中您并不总是会看到 2 倍的加速。这只能在 UICA 或 nanobench 等理想化环境中实现。在实际应用程序中，即使您最大化了 FMA 的执行吞吐量，收益也可能会受到最终缓存未命中和其他流水线冲突的阻碍。发生这种情况时，缓存未命中的影响会超过 FMA 端口利用率不理想的影响。这很容易导致令人失望的 5% 速度提升。但别担心，你仍然做对了。

最后，让我们提醒您，UICA 或任何其他静态性能分析器都不适合分析大段代码。但它们非常适合探索微架构效应。此外，它们还可以帮助您建立 CPU 工作方式的心理模型。UICA 的另一个非常重要的用例是在循环中找到关键依赖性链，正如 easyperf 博客的 文章: https://easyperf.net/blog/2022/05/11/Visualizing-Performance-Critical-Dependency-Chains[80] 中所述。

5.8 编译器优化报告

如今，软件开发在很大程度上依赖编译器进行性能优化。编译器在加速软件方面扮演着关键角色。大多数开发人员将优化代码的工作留给编译器，只有当他们发现编译器无法完成的优化机会时才会干预。可以说，这是一个好的默

[80] Easyperf 博客 - https://easyperf.net/blog/2022/05/11/Visualizing-Performance-Critical-Dependency-Chains

认策略。但是，当您追求最佳性能时，它就不太管用了。如果编译器没有执行关键优化，例如向量化循环，怎么办？您将如何知道这一点？幸运的是，所有主流编译器都提供优化报告，我们现在将讨论这些报告。

假设您想知道一个关键循环是否被展开。如果是，展开因子是多少？有一种艰苦的方法可以知道这一点：研究生成的汇编指令。不幸的是，并不是每个人都习惯于阅读汇编语言。如果函数很大，它调用其他函数或也有许多被向量化的循环，或者如果编译器为同一个循环创建了多个版本，这可能会特别困难。大多数编译器，包括 GCC、Clang 和 Intel 编译器（但不包括 MSVC），都提供优化报告，用于检查特定代码段执行了哪些优化。

让我们看一下 Listing 5.8，它展示了一个由 clang 16.0 未向量化的循环示例。

代码清单：a.c

```
1 void foo(float* __restrict__ a,
2          float* __restrict__ b,
3          float* __restrict__ c,
4          unsigned N) {
5   for (unsigned i = 1; i < N; i++) {
6     a[i] = c[i-1]; // value is carried over from previous iteration
7     c[i] = b[i];
8   }
9 }
```

在 clang 中生成优化报告，您需要使用 -Rpass*: https://llvm.org/docs/Vectorizers.html#diagnostics 标志：

```
$ clang -O3 -Rpass-analysis=.* -Rpass=.* -Rpass-missed=.* a.c -c
a.c:5:3: remark: loop not vectorized [-Rpass-missed=loop-vectorize]
  for (unsigned i = 1; i < N; i++) {
  ^
a.c:5:3: remark: unrolled loop by a factor of 8 with run-time trip count [-Rpass=loop-unroll]
  for (unsigned i = 1; i < N; i++) {
  ^
```

检查上面的优化报告，我们可以看到循环没有被向量化，而是被展开了。开发人员并不总是很容易识别 Listing 5.8 第 6 行循环中是否存在循环进位依赖。由 c[i-1] 加载的值取决于前一次迭代的存储（参见图 37 中的操作 ② 和 ③）。可以通过手动展开循环的前几个迭代来揭示依赖关系：

```
// iteration 1
  a[1] = c[0];
  c[1] = b[1]; // writing the value to c[1]
// iteration 2
  a[2] = c[1]; // reading the value of c[1]
  c[2] = b[2];
...
```

如果我们将 Listing 5.8 中的代码向量化，它会导致在数组 a 中写入错误的值。假设 CPU SIMD 单元可以一次处理四个浮点数，我们可以得到可以用以下伪代码表示的代码：

```
// iteration 1
  a[1..4] = c[0..3]; // oops!, a[2..4] get wrong values
  c[1..4] = b[1..4];
...
```

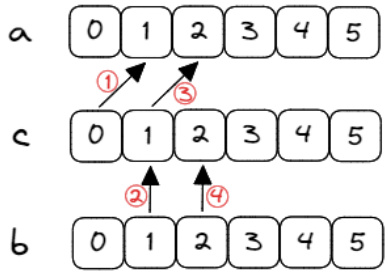


Figure 37: 在 Listing 5.8中可视化操作顺序

Listing 5.8 中的代码无法向量化，因为循环内部的操作顺序很重要。如 Listing 5.8 所示，通过交换第 6 行和第 7 行可以修复此示例。这不会改变代码的语义，所以这是一个完全合法的更改。另外，可以通过将循环拆分成两个单独的循环来改善代码。

代码清单：a.c

```
1 void foo(float* __restrict__ a,
2          float* __restrict__ b,
3          float* __restrict__ c,
4          unsigned N) {
5   for (unsigned i = 1; i < N; i++) {
6     c[i] = b[i];
7     a[i] = c[i-1];
8   }
9 }
```

在优化报告中，我们可以看到循环成功向量化了：

```
$ clang -O3 -Rpass-analysis=.* -Rpass=.* -Rpass-missed=.* a.c -c
a.cpp:5:3: remark: vectorized loop (vectorization width: 8, interleaved count: 4)
    [-Rpass=loop-vectorize]
  for (unsigned i = 1; i < N; i++) {
  ^
```

这只是使用优化报告的一个例子，我们将在本书的第二部分讨论发现向量化机会时更详细地介绍。编译器优化报告可以帮助您找到错过的优化机会，并了解这些机会错过的原因。此外，编译器优化报告对于测试假设很有用。编译器通常会根据其成本模型分析来决定某个转换是否有益。但编译器并不总是做出最佳选择。一旦您在报告中发现缺少关键优化，您可以尝试通过更改源代码或向编译器提供提示（例如 #pragma、属性、编译器内置函数等）来纠正它。始终通过在实际环境中进行测量来验证您的假设。

编译器报告可能相当庞大，每个源代码文件都会生成单独的报告。有时，在输出文件中找到相关记录可能成为一项挑战。我们应该提到，最初这些报告的设计明确供编译器编写者用于改进优化过程。多年来，已经出现了一些工具，使它们更易于应用程序开发人员访问和操作。最值得注意的是 opt-viewer[81] 和 optview2[82]。此外，Compiler Explorer 网站[83]还为基于 LLVM 的编译器提供了“优化输出”工具，当您将鼠标悬停在源代码相应行上时，它会报告执行的转换。所有这些工具都帮助可视化基于 LLVM 的编译器成功的和失败的代码转换。

[81] opt-viewer - https://github.com/llvm/llvm-project/tree/main/llvm/tools/opt-viewer

[82] optview2 - https://github.com/OfekShilon/optview2

[83] Compiler Explorer - https://godbolt.org/

在 LTO[84] 模式下，一些优化是在链接阶段进行的。为了同时从编译和链接阶段发出编译器报告，应该向编译器和链接器传递专用选项。有关更多信息，请参见 LLVM "remarsk"[85]指南。

Intel® ISPC[86] 编译器 (已在 Section 9.3.7.5 中讨论) 采用稍微不同的方式报告缺失的优化。它会针对编译为相对低效代码的代码结构发出警告。无论哪种方式，编译器优化报告都应该是您工具箱中的关键工具之一。它是一种快速的方法，可以检查对特定热点进行了哪些优化，以及是否失败了一些重要的优化。许多改进机会都是通过编译器优化报告发现的。

问题和练习

1. 在以下场景中你会使用哪些方法?

   - 场景 1：客户支持团队报告客户问题：升级到新版本应用程序后，特定操作的性能下降了 10%。
   - 场景 2：客户支持团队报告客户问题：某些交易完成时间比平时长 2 倍，没有特定模式。
   - 场景 3：您正在评估三种不同的压缩算法，想知道每种算法都存在哪些类型的性能瓶颈（内存延迟/带宽、分支预测错误等）。
   - 场景 4：有一个新的闪亮库声称比您目前项目中集成的库更快；您决定比较它们的性能。
   - 场景 5：您被要求分析不熟悉代码的性能；想知道某个分支被采取的频率以及循环执行的迭代次数。

   AI 解答：在这些场景中，我会采用以下方法：

   - 场景 1：性能下降问题
     - 性能比较：首先，我会使用基准测试工具（如 Linux Perf 或其他性能分析工具）来比较新旧版本在相同条件下的性能。
     - 代码插桩：如果需要更详细的信息，我会在关键代码区域进行代码插桩，以收集特定运行时信息。
     - 跟踪：使用系统调用跟踪工具（如 strace）来监控应用程序的行为，以便发现可能的系统资源瓶颈。
     - 采样：进行性能采样以识别性能下降的关键热点。
   - 场景 2：交易完成时间异常
     - 跟踪：使用跟踪工具来记录交易完成过程中的系统调用和事件，以便分析延迟的原因。
     - 采样：进行性能采样，特别关注交易处理期间的性能数据。
     - 工作负载特征化：分析交易处理的工作负载特征，以确定是否存在异常模式。
   - 场景 3：评估压缩算法性能瓶颈
     - Roofline 性能模型：使用 Roofline 模型来评估每种算法的性能，并识别可能的瓶颈（如内存带宽或计算速度）。
     - 性能计数器：使用性能计数器来收集每种算法的详细性能数据，如缓存未命中、分支预测错误等。
     - 静态性能分析：对算法的汇编代码进行静态分析，以预测其在真实硬件上的行为。
   - 场景 4：比较新旧库性能
     - 基准测试：设计和运行基准测试来比较新旧库的性能。
     - 性能采样：使用性能采样工具来收集新旧库在执行时的性能数据。
     - 编译器优化报告：检查编译器优化报告，了解是否有优化被遗漏或可以改进的地方。
   - 场景 5：分析不熟悉的代码
     - 代码插桩：在关键的代码路径上进行代码插桩，以收集执行频率和迭代次数等信息。
     - 跟踪：使用跟踪工具来监控程序的执行流程，帮助理解代码结构和逻辑。
     - 性能分析工具：使用性能分析工具（如 Linux Perf）来识别热点和性能瓶颈。

[84] 链接时间优化，也称为过程间优化 (IPO)。阅读更多: https://en.wikipedia.org/wiki/Interprocedural_optimization

[85] LLVM compiler remarks - https://llvm.org/docs/Remarks.html

[86] ISPC - https://ispc.github.io/ispc.html

在所有这些场景中，我会结合使用多种性能分析方法，以确保从不同角度全面理解性能问题，并找到最有效的优化策略。

2. 运行您每天使用的应用程序。使用本章讨论的方法进行性能分析练习。收集各种 CPU 性能事件的原始计数，找到热点，收集 roofline 数据，并生成和研究程序中热点函数的编译器优化报告。

章节总结

- 延迟和吞吐量通常是程序性能的最终指标。当寻求改善它们的方法时，我们需要获取更多关于应用程序如何执行的详细信息。硬件和软件都提供可用于性能监视的数据。
- 代码检测允许我们跟踪程序中的许多内容，但会在开发和运行时都造成相对较大的开销。虽然现在开发人员很少手动检测他们的代码，但这种方法仍然与自动化流程相关，例如 PGO。
- 跟踪在概念上类似于检测，可用于探索系统中的异常。跟踪允许我们捕获整个事件序列，并在每个事件上附加时间戳。
- 工作负载特征化是一种基于应用程序运行时行为进行比较和分组的方法。一旦进行特征化，就可以遵循特定的方法来找到程序中的优化空间。带有标记 API 的分析工具可用于分析特定代码区域的性能。
- 采样跳过程序执行的大部分，只获取一个样本，该样本应该代表整个区间。尽管如此，采样通常会产生足够精确的分布。采样最著名的用例是找到代码中的热点。采样是最流行的分析方法，因为它不需要重新编译程序，并且运行时开销非常小。
- 通常，计数和采样会产生非常低的运行时开销（通常低于 2%）。一旦您开始在不同事件之间进行多路复用，计数就会变得更加昂贵（5-15% 的开销），采样会随着采样频率的增加而变得更加昂贵 [Nowak & Bitzes, 2014]。考虑使用用户模式采样来分析长时间运行的工作负载或您不需要非常准确的数据时。
- Roofline 性能模型是一个面向吞吐量的性能模型，在高性能计算 (HPC) 领域得到了广泛使用。它允许绘制应用程序性能与硬件限制之间的关系图。Roofline 模型有助于识别性能瓶颈，指导软件优化，并跟踪优化进度。
- 有些工具试图静态分析代码的性能。此类工具模拟一段代码而不是执行它。这种方法存在许多限制和约束，但您会得到非常详细和低级别的报告作为回报。
- 编译器优化报告有助于发现丢失的编译器优化。这些报告还指导开发人员设计新的性能实验。